

UACM

Universidad Autónoma de la Ciudad de México

NADA HUMANO ME ES AJENO

ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

PROFESOR:

EMMANUEL VITE SEVILLA

Sistema de Autenticación de Dos Pasos (2FA)

Entrega 1:

Definición de entorno, estándares y selección de herramientas

ALUMNOS:

CRUZ MEJIA OSCAR ADOLFO

MARES PACHECO LAURA SARAI

MATRÍCULA:

23-011-0820

23-011-1462

1. Descripción del proyecto 3
2. Selección del ciclo de vida o modelo de desarrollo 3
2.1 Modelo seleccionado: Desarrollo iterativo e incremental 3
2.2 Justificación 3
Organización del desarrollo 4
2.2 Organización por ciclos 4
2.3 Ciclo 1: Planeación y requerimientos 4
3. Estándares de documentación, diseño y codificación 4
3.1 Estándares de documentación 4
3.2 Estándares de diseño 5
3.3 Estándares de codificación 6
4. Selección de herramientas de pruebas 7
4.1 JUnit: pruebas unitarias 7
4.2 Selenium: pruebas funcionales automatizadas 8
4.3 Relación entre las herramientas 9
5. Asignación de roles del equipo 9
Responsabilidad transversal 10
6. Bitácora inicial de trabajo 10
6.1 Registro de actividades 10
6.2 Acuerdos del equipo 11
6.3 Control de cambios 11
7. Identificación de riesgos iniciales 11
7.1 Seguimiento de riesgos 12
8. Alcance de la primera versión del sistema 12
9. Conclusión 13

1. Descripción del proyecto

El proyecto consiste en desarrollar y evaluar un Sistema de Autenticación de Dos Pasos (2FA), aplicando buenas prácticas de ingeniería de software y aseguramiento de la calidad.

El sistema permitirá que un usuario se autentique mediante dos etapas:

1. Ingresar su nombre de usuario y contraseña.
2. Contestar correctamente dos preguntas de seguridad previamente registradas.

El acceso será autorizado únicamente cuando ambas etapas sean satisfactorias.

El proyecto utiliza Java, JSP y Servlets, bajo el patrón Modelo–Vista–Controlador (MVC), y el patrón DAO para separar las operaciones de acceso a datos de la lógica de la aplicación.

El objetivo no es solamente obtener un sistema funcional, sino también documentar, revisar y comprobar la calidad de sus requerimientos, diseño, código y pruebas.

2. Selección del ciclo de vida o modelo de desarrollo

2.1 Modelo seleccionado: Desarrollo iterativo e incremental

Se seleccionó el modelo iterativo e incremental porque permite desarrollar el sistema por etapas, revisar los resultados y realizar correcciones antes de continuar con las siguientes actividades.

Este modelo se adapta al proyecto porque el sistema tiene un flujo de autenticación dividido en dos etapas. Primero se debe comprobar el acceso mediante usuario y contraseña; después se verifica la identidad mediante las preguntas de seguridad.

Además, permite integrar progresivamente las actividades de documentación, diseño, programación y pruebas, facilitando la detección temprana de defectos.

2.2 Justificación

Se eligió este modelo por las siguientes razones:

- El sistema puede dividirse en funcionalidades pequeñas y verificables.
- Permite revisar cada avance antes de continuar.
- Facilita la identificación y corrección de errores.
- Permite incorporar las pruebas unitarias y funcionales durante el desarrollo.
- Ayuda a mantener actualizada la documentación conforme evoluciona el sistema.
- Facilita el seguimiento de las actividades y responsabilidades de cada integrante.

Organización del desarrollo

Etapa	Actividades	Resultado esperado
Planeación	Definir el ciclo de vida, estándares, herramientas, roles y riesgos.	Plan inicial del proyecto.
Requerimientos	Identificar y documentar los requisitos funcionales y no funcionales.	Documento de requerimientos.
Diseño	Definir la arquitectura MVC, DAO y diagramas UML.	Diseño del sistema.
Desarrollo	Implementar y revisar las funcionalidades del sistema.	Código fuente integrado.
Pruebas	Ejecutar pruebas unitarias con JUnit y funcionales con Selenium.	Resultados y registro de defectos.
Evaluación	Revisar resultados, métricas y cumplimiento de requisitos.	Informe final de calidad.

2.2 Organización por ciclos

2.3 Ciclo 1: Planeación y requerimientos

Se trabajará en la definición del proyecto, estándares, herramientas, organización del equipo, riesgos y documentación de los requerimientos.

2.4 Ciclo 2: Diseño, desarrollo y evaluación

Se trabajará en el diseño de la arquitectura, implementación, pruebas, corrección de defectos y evaluación de la calidad del sistema.

Las cuatro entregas del curso servirán como puntos de revisión del avance.

3. Estándares de documentación, diseño y codificación

Para mantener la calidad y uniformidad del proyecto se establecen los siguientes estándares. Cada uno incluye reglas que pueden comprobarse durante las revisiones.

3.1 Estándares de documentación

ID	Regla	Forma de verificación
DOC-01	Cada documento debe tener título, nombre del proyecto, fecha y versión.	Revisar los encabezados de los documentos.
DOC-02	Los requisitos deben tener identificadores únicos.	Comprobar que no existan identificadores repetidos.
DOC-	Los casos de uso deben incluir actor,	Revisar cada caso de uso.

03	precondiciones, flujo normal y alternativo, y postcondiciones.	
DOC-04	Los diagramas deben tener título y nombres comprensibles.	Inspeccionar los diagramas UML.
DOC-05	Los defectos y cambios deben registrarse con una descripción y su estado.	Revisar el registro de defectos y cambios.
DOC-06	La documentación debe mantenerse actualizada conforme se modifique el sistema.	Comparar los documentos con la versión del código.

Ejemplo:

Un requisito debe identificarse como RF-01, tener un nombre y describir una función concreta.

Ejemplo:

RF-01. Inicio de sesión: El sistema debe permitir que el usuario ingrese su nombre de usuario y contraseña para iniciar la primera etapa de autenticación.

3.2 Estándares de diseño

ID	Regla	Forma de verificación
DIS-01	El sistema debe mantener la separación de responsabilidades del patrón MVC.	Revisar la ubicación y responsabilidad de las clases.
DIS-02	Las vistas se encargarán de mostrar la información y recibir los datos del usuario.	Revisar los archivos JSP.
DIS-03	Los controladores procesarán las solicitudes y coordinarán el flujo de la aplicación.	Revisar los Servlets.
DIS-04	El modelo contendrá las clases y la lógica correspondiente a los datos y reglas del sistema.	Revisar las clases del paquete modelo.
DIS-05	El acceso a datos debe estar separado mediante el patrón DAO.	Comprobar que las operaciones de persistencia se concentren en las clases DAO.
DIS-06	Los diagramas UML deben representar los elementos y relaciones principales del sistema.	Comparar los diagramas con la implementación.

Ejemplo de organización:

- **Modelo:** Usuario, UsuarioDAO y Seguridad.
- **Controlador:** LoginServlet, PreguntasServlet y VerificarPreguntasServlet.
- **Vista:** index.jsp, preguntas.jsp y exito.jsp.

3.3 Estándares de codificación

Se utilizarán convenciones de programación para Java, manteniendo nombres claros, código legible y separación de responsabilidades.

ID	Regla	Forma de verificación
COD-01	Las clases utilizarán nombres en PascalCase.	Revisar los nombres de las clases.
COD-02	Los métodos y variables utilizarán nombres en camelCase.	Revisar los identificadores.
COD-03	Las constantes se escribirán con letras mayúsculas y guion bajo cuando corresponda.	Revisar las declaraciones de constantes.
COD-04	Cada clase tendrá una responsabilidad claramente definida.	Revisar que no concentre funciones ajenas a su propósito.
COD-05	Se evitará repetir código cuando pueda reutilizarse una función o método.	Realizar inspecciones de código.
COD-06	Se validarán los datos recibidos desde los formularios.	Revisar los controladores y las validaciones.
COD-07	Se utilizarán nombres descriptivos para variables, métodos y clases.	Inspeccionar el código fuente.
COD-08	Se manejarán adecuadamente las sesiones y los errores de autenticación.	Revisar el flujo de los Servlets.
COD-09	Los cambios importantes deberán registrarse en el repositorio.	Revisar el historial de versiones.

Ejemplo de codificación:

```
public boolean compararRespuestas(String respuesta1, String respuesta2) {
   if (respuesta1 == null || respuesta2 == null) {
      return false;
   }

   return respuesta1.trim().equalsIgnoreCase(respuesta2.trim());
}
```

Este ejemplo utiliza un nombre descriptivo, valida valores nulos y mantiene una operación concreta dentro del método.

4. Selección de herramientas de pruebas

Para comprobar la calidad del sistema se seleccionan dos herramientas principales: JUnit y Selenium.

4.1 JUnit: pruebas unitarias

Herramienta seleccionada: JUnit 5.

Objetivo: Comprobar individualmente el funcionamiento de los métodos y componentes que puedan probarse de forma aislada.

Se utilizará principalmente para revisar la lógica del modelo y las funciones de seguridad.

Componentes candidatos a pruebas:

- Seguridad: comprobar la comparación de respuestas.
- UsuarioDAO: comprobar la validación de credenciales y la recuperación de usuarios.
- Usuario: comprobar el almacenamiento y recuperación de sus atributos.

Ejemplos de pruebas previstas:

ID	Prueba	Resultado esperado
JU-01	Comparar dos respuestas iguales.	El resultado debe ser verdadero.
JU-02	Comparar respuestas con mayúsculas y minúsculas diferentes.	El resultado debe ser verdadero.
JU-03	Comparar respuestas diferentes.	El resultado debe ser falso.
JU-04	Comparar respuestas vacías o nulas.	El método debe manejar la entrada sin producir un error inesperado.
JU-05	Validar credenciales correctas.	Se debe recuperar el usuario correspondiente.
JU-06	Validar credenciales incorrectas.	No se debe autorizar al usuario.

Configuración prevista:

I. Utilizar Eclipse como entorno de desarrollo.
II. Incorporar la biblioteca JUnit 5 al proyecto.
III. Crear las clases de prueba correspondientes.
IV. Ejecutar las pruebas desde Eclipse mediante la opción de ejecución JUnit.
V. Registrar los resultados obtenidos y los defectos encontrados.

Estado: Herramienta seleccionada. La integración de la biblioteca y la ejecución de las pruebas deberán comprobarse y documentarse con evidencia.

4.2 Selenium: pruebas funcionales automatizadas

Herramienta seleccionada: Selenium WebDriver.

Objetivo: Automatizar las interacciones del usuario con la interfaz web para comprobar el funcionamiento del proceso de autenticación.

Se utilizará para verificar los formularios, las respuestas del sistema y la navegación entre las páginas.

Escenarios candidatos a pruebas:

ID	Prueba	Resultado esperado
SE-01	Ingresar usuario y contraseña correctos.	El sistema debe mostrar la etapa de preguntas de seguridad.
SE-02	Ingresar credenciales incorrectas.	El sistema debe rechazar el acceso.
SE-03	Contestar correctamente las dos preguntas.	El sistema debe permitir el acceso.
SE-04	Contestar incorrectamente una pregunta.	El sistema debe rechazar la autenticación.
SE-05	Intentar acceder directamente a la página de éxito sin autenticarse.	El sistema debe redirigir al inicio de sesión.
SE-06	Cerrar la sesión.	El sistema debe finalizar la sesión del usuario.

Configuración prevista:

I. Utilizar Selenium WebDriver.
II. Incorporar las bibliotecas necesarias al proyecto de pruebas.
III. Utilizar un navegador compatible, como Google Chrome.
IV. Preparar el controlador del navegador y la configuración necesaria para ejecutarlo.
V. Crear los casos de prueba automatizados.
VI. Ejecutar las pruebas contra la aplicación desplegada en Tomcat.
VII. Guardar los resultados y evidencias de ejecución.

Estado: Herramienta seleccionada. La integración, configuración del navegador y ejecución deberán comprobarse antes de registrar las pruebas como realizadas.

4.3 Relación entre las herramientas

Herramienta	Tipo de prueba	Elementos que revisa
JUnit 5	Unitaria	Métodos y componentes individuales del modelo.
Selenium WebDriver	Funcional automatizada	Formularios, navegación y flujo completo de autenticación.

Las dos herramientas se complementan: JUnit permite revisar la lógica interna y Selenium permite comprobar el comportamiento del sistema desde la perspectiva del usuario.

5. Asignación de roles del equipo

El equipo distribuirá las responsabilidades de acuerdo con los cuatro roles establecidos en la guía del proyecto.

Rol	Integrante responsable	Responsabilidades
Programador principal (líder técnico)	[Cruz Mejia Oscar Adolfo]	Coordinar la integración técnica, mantener la estructura del sistema, atender bloqueos y comunicar avances.
Encargado de pruebas (QA)	[Cruz Mejia Oscar Adolfo]	Preparar pruebas, listas de revisión, organizar revisiones entre compañeros y dar seguimiento a defectos.
Documentador (gestión de configuración)	[Mares Pacheco Laura Sarai]	Mantener la documentación, registrar versiones, administrar cambios y comunicar modificaciones.
Integrador (procesos y calidad)	[Mares Pacheco Laura Sarai]	Dar seguimiento al plan, registrar riesgos y verificar el cumplimiento de los entregables.

Responsabilidad transversal

Todos los integrantes participarán en el diseño, implementación, revisión y corrección del código.

La asignación de un rol principal no significa que una persona sea la única responsable de esa actividad. El equipo deberá colaborar en las diferentes etapas del desarrollo.

6. Bitácora inicial de trabajo

La bitácora permitirá registrar las actividades realizadas, los responsables, el tiempo invertido y los acuerdos del equipo.

6.1 Registro de actividades

Fecha	Actividad	Responsable	Tiempo invertido (Hrs)	Resultado
30/10/2026	Revisión de las instrucciones del proyecto.	Todo el equipo	1	Identificación de las actividades de la Entrega 1.
02/11/2026	Selección del modelo de desarrollo.	Todo el equipo	[Tiempo real]	Modelo iterativo e incremental seleccionado.
03/11/2026	Definición de estándares.	Todo el equipo	[Tiempo real]	Reglas de documentación, diseño y codificación.
04/11/2026	Selección de herramientas de pruebas.	Cruz Mejia Oscar Adolfo	[Tiempo real]	JUnit 5 y Selenium WebDriver seleccionados.
05/11/2026	Asignación de roles.	Todo el equipo	[Tiempo real]	Responsabilidades distribuidas.
05/11/2026	Identificación de riesgos iniciales.	Mares Pacheco Laura Sarai	[Tiempo real]	Riesgos y acciones de mitigación documentados.

6.2 Acuerdos del equipo

I. Mantener comunicación entre los integrantes durante el desarrollo.
II. Respetar las responsabilidades asignadas.
III. Registrar las actividades y acuerdos importantes en la bitácora.
IV. Revisar los cambios antes de integrarlos al proyecto.
V. Mantener actualizado el repositorio compartido.
VI. Comunicar los problemas que puedan afectar el avance.
VII. Participar en las actividades de desarrollo y revisión del código.
VIII. Registrar los defectos encontrados y dar seguimiento a su corrección.

6.3 Control de cambios

Los cambios realizados al proyecto deberán identificarse mediante el historial del repositorio.

Cada modificación importante deberá incluir una descripción clara de lo que se modificó y su propósito.

Ejemplo de mensaje de cambio:

Agrega validación de respuestas de seguridad

7. Identificación de riesgos iniciales

Se identificaron riesgos que pueden afectar el desarrollo, la organización y la calidad del sistema.

ID	Riesgo	Probabilidad	Impacto	Estrategia de mitigación
R-01	Requerimientos ambiguos o incompletos.	Media	Alto	Revisar los requisitos, aclarar dudas y utilizar identificadores únicos.
R-02	Estimaciones de tiempo poco realistas.	Media	Alto	Dividir las actividades en tareas pequeñas y revisar periódicamente el avance.
R-03	Comunicación insuficiente entre integrantes.	Media	Alto	Mantener reuniones y registrar acuerdos en la bitácora.
R-04	Cambios no controlados en el código.	Media	Alto	Utilizar el repositorio compartido y registrar las modificaciones.
R-05	Problemas al integrar JUnit y Selenium.	Alta	Medio	Realizar pruebas de configuración anticipadas y documentar los problemas.
R-06	Defectos en el proceso de	Media	Alto	Revisar el flujo de inicio de sesión y las preguntas de

	autenticación.			seguridad mediante pruebas.
R-07	Conflictos al integrar cambios de diferentes integrantes.	Media	Medio	Revisar los cambios antes de integrarlos y mantener comunicación.
R-08	Retraso en las actividades de documentación.	Media	Medio	Asignar un responsable y actualizar los documentos conforme avance el proyecto.

7.1 Seguimiento de riesgos

Los riesgos se revisarán durante las reuniones del equipo y se actualizará su estado conforme avance el proyecto.

Se utilizarán los siguientes estados:

- **Abierto:** el riesgo fue identificado y todavía requiere seguimiento.
- **En seguimiento:** se están aplicando acciones para reducirlo.
- **Mitigado:** las acciones aplicadas redujeron su impacto o probabilidad.
- **Materializado:** el riesgo ocurrió y debe registrarse como problema o defecto cuando corresponda.

8. Alcance de la primera versión del sistema

La primera versión del sistema contempla las siguientes funcionalidades:

I. Inicio de sesión mediante usuario y contraseña.
II. Validación de las credenciales.
III. Presentación de dos preguntas de seguridad.
IV. Validación de las respuestas proporcionadas.
V. Autorización del acceso cuando ambas etapas sean correctas.
VI. Rechazo del acceso cuando las credenciales o respuestas sean incorrectas.
VII. Protección de las páginas que requieren autenticación.
VIII. Cierre de sesión.

Las mejoras opcionales de envío de PIN por correo electrónico y generación de códigos TOTP no forman parte del alcance obligatorio inicial.

9. Conclusión

En esta primera entrega se estableció la forma en que se organizará y desarrollará el Sistema de Autenticación de Dos Pasos. Se seleccionó el modelo iterativo e incremental porque permite trabajar por etapas, revisar los avances y corregir errores durante el desarrollo.

También se definieron estándares para mantener ordenada la documentación, el diseño y el código, además de seleccionar JUnit y Selenium como herramientas para las pruebas. Finalmente, se establecieron los roles del equipo, el inicio de la bitácora y los principales riesgos que podrían afectar el proyecto.

Con esta organización se busca que el sistema no solamente funcione correctamente, sino que también pueda demostrar que fue desarrollado y revisado siguiendo criterios de calidad.